5.1.1 - who are the people behind the events?

```
---------
-- 5.1 --
---------

-- "Who are the people behind the events?"
-- Translated as: Which Users LIKED Which Posts, and When?
select users.user_name, users.email, users.signup_date, 'LIKED' as event,
       posts.post_title, posts.posted_on, likes.liked_at, posts.author_country, users.country as liker_country
from users
inner join likes on users.user_id = likes.user_id
inner join posts on likes.post_id = posts.post_id
order by users.user_name, likes.like_id;

---------
```

Data Output | Messages | Notifications

Showing rows: 1 to 200 | Page No: 1 of 1

	user_name character varying (120)	email character varying (160)	signup_date date	event text	post_title character varying (120)	posted_on date	liked_at timestamp without time zone	author_country character varying (80)	liker_country character varying (80)
1	User_01	user01@example.com	2022-03-18	LIKED	Post_01	2022-07-15	2023-01-01 15:48:00	India	Brazil
2	User_01	user01@example.com	2022-03-18	LIKED	Post_02	2022-01-20	2023-12-17 07:13:00	Vietnam	Brazil
3	User_01	user01@example.com	2022-03-18	LIKED	Post_03	2022-05-15	2023-07-06 19:25:00	Brazil	Brazil
4	User_01	user01@example.com	2022-03-18	LIKED	Post_03	2022-05-15	2023-12-18 12:49:00	Brazil	Brazil
5	User_01	user01@example.com	2022-03-18	LIKED	Post_06	2022-03-10	2023-08-23 18:44:00	Vietnam	Brazil
6	User_01	user01@example.com	2022-03-18	LIKED	Post_06	2022-03-10	2023-03-18 23:27:00	Vietnam	Brazil
7	User_01	user01@example.com	2022-03-18	LIKED	Post_08	2022-04-17	2023-02-08 15:57:00	India	Brazil
8	User_01	user01@example.com	2022-03-18	LIKED	Post_09	2022-04-25	2023-09-19 05:02:00	UK	Brazil
9	User_01	user01@example.com	2022-03-18	LIKED	Post_09	2022-04-25	2023-05-22 15:28:00	UK	Brazil
10	User_01	user01@example.com	2022-03-18	LIKED	Post_10	2022-05-02	2023-11-26 06:51:00	US	Brazil
11	User_01	user01@example.com	2022-03-18	LIKED	Post_10	2022-05-02	2023-01-14		

✓ Successfully run. Total query runtime: 97 msec. 200 rows affected.

Total rows: 200 | Query complete 00:00:00.097 | CRLF | Ln 14, Col 1

5.1.2 - producers by “catalog credentials”

```
---------
-- 5.1.2 --
---------

-- "Return the producers associated with your theme's catalog tag"
-- Translated as: What hashtags are associated to each Post?
select hashtags.hashtag_name, posts.post_title, posts.post_id
from posts
inner join post_hashtags on posts.post_id = post_hashtags.post_id
inner join hashtags on post_hashtags.hashtag_id = hashtags.hashtag_id
order by hashtag_name, post_id;
                --- VIEW it by the "post side of the equation"
select  posts.post_id, posts.post_title, hashtags.hashtag_name
from posts
inner join post_hashtags on posts.post_id = post_hashtags.post_id
inner join hashtags on post_hashtags.hashtag_id = hashtags.hashtag_id
order by post_id, hashtag_name;
```

Data Output | Messages | Notifications

Showing rows: 1 to 25 | Page No: 1 of 1

	post_id integer	post_title character varying (120)	hashtag_name character varying (60)
1	1	Post_01	food
2	1	Post_01	travel
3	2	Post_02	fitness
4	2	Post_02	tech
5	2	Post_02	travel
6	3	Post_03	food
7	3	Post_03	tech
8	4	Post_04	food
9	4	Post_04	music
10	4	Post_04	travel
11	5	Post_05	tech
12	5	Post_05	travel
13	6	Post_06	fitness

Total rows: 25 | Query complete 00:00:00.085 | CRLF | Ln 41, Col 1

5.1.3 - how do the producers compare with one another? self join

```sql
---------
-- 5.1.3 --
---------

-- "How to the producers compare with one another?"
-- Translated as: for each post, how many likes does it have? And how many does it have compared to every other post?
-- NOTE: EVERY OTHER means use the "<>" operator. If I wanted unique pairs I would have used "<"; but I'm ok with the "duplicates"

-- There is a LIKE COUNT in the POSTS table, so we can do a "true self join here"
select a.post_title, a.like_count, b.post_title, b.like_count, a.like_count - b.like_count as difference
from posts as a
inner join posts as b on a.post_id<>b.post_id
order by a.post_title, b.post_title;


-- the LIKE COUNTS from this table are different than the LIKE counts if we summed up on the LIKES table
```

Data Output | Messages | Notifications

Showing rows: 1 to 210 | Page No: 1 of 1

	post_title character varying (120)	like_count integer	post_title character varying (120)	like_count integer	difference integer
1	Post_01	400	Post_02	400	0
2	Post_01	400	Post_03	400	0
3	Post_01	400	Post_04	400	0
4	Post_01	400	Post_05	400	0
5	Post_01	400	Post_06	0	400
6	Post_01	400	Post_07	0	400
7	Post_01	400	Post_08	336	64
8	Post_01	400	Post_09	381	19
9	Post_01	400	Post_10	337	63
10	Post_01	400	Post_11	207	193
11	Post_01	400	Post_12	51	349
12	Post_01	400	Post_13	153	247
13	Post_01	400	Post_14	129	271

Total rows: 210 | Query complete 00:00:00.117 | CRLF | Ln 42, Col 1

5.2.A - left join

```sql
-- PART A :
-----------

-- Use LEFT JOIN to return all producers, including those who have never had an event.

select posts.post_id, posts.post_title, count(likes.like_id)
from posts
left join likes on likes.post_id = posts.post_id
group by posts.post_id
order by posts.post_id;


-- Use an INNER JOIN

select posts.post_id, posts.post_title, count(likes.like_id)
from posts
inner join likes on likes.post_id = posts.post_id
group by posts.post_id
order by posts.post_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 15

	post_id [PK] integer	post_title character varying (120)	count bigint
2	2	Post_02	16
3	3	Post_03	13
4	4	Post_04	6
5	5	Post_05	18
6	6	Post_06	21
7	7	Post_07	18
8	8	Post_08	20
9	9	Post_09	14
10	10	Post_10	19
11	11	Post_11	16
12	12	Post_12	17
13	13	Post_13	11
14	14	Post_14	0
15	15	Post_15	0

Total rows: 15 | Query complete 00:00:00.101

5.2.A - inner join

```sql
-- Use an INNER JOIN

select posts.post_id, posts.post_title, count(likes.like_id)
from posts
inner join likes on likes.post_id = posts.post_id
group by posts.post_id
order by posts.post_id;

-- "Compare the Results"
Select 'The Inner Join missed Post_14 and Post_15';

-- confirm using NOT IN with a safety measure
select count(post_id) from posts
where post_id NOT IN (select post_id from likes where post_id is not null);


-----------
```

Data Output | Messages | Notifications

Showing rows

	post_id [PK] integer	post_title character varying (120)	count bigint
1	1	Post_01	11
2	2	Post_02	16
3	3	Post_03	13
4	4	Post_04	6
5	5	Post_05	18
6	6	Post_06	21
7	7	Post_07	18
8	8	Post_08	20
9	9	Post_09	14
10	10	Post_10	19
11	11	Post_11	16
12	12	Post_12	17
13	13	Post_13	11

5.2.B - Right join

```
38
39  -----------
40  -- PART B :
41  -----------
42
43  -- Write a RIGHT JOIN that returns the same records as your LEFT JOIN from Part A.
44  select posts.post_id, posts.post_title, count(likes.like_id)
45  from posts
46  right join likes on likes.post_id = posts.post_id
47  group by posts.post_id
48  order by posts.post_id;
49
50  -- confirm that they match
```

Data Output | Messages | Notifications

Showing rows: 1 to 13

	post_id [PK] integer	post_title character varying (120)	count bigint
1	1	Post_01	11
2	2	Post_02	16
3	3	Post_03	13
4	4	Post_04	6
5	5	Post_05	18
6	6	Post_06	21
7	7	Post_07	18
8	8	Post_08	20
9	9	Post_09	14
10	10	Post_10	19
11	11	Post_11	16
12	12	Post_12	17
13	13	Post_13	11

5.2.B - full outer join

```sql
-- full outer join
select posts.post_id, posts.post_title, count(likes.like_id)
from posts
full outer join likes on likes.post_id = posts.post_id
group by posts.post_id
order by posts.post_id;
```

Data Output | Messages | Notifications

	post_id [PK] integer	post_title character varying (120)	count bigint
1	1	Post_01	11
2	2	Post_02	16
3	3	Post_03	13
4	4	Post_04	6
5	5	Post_05	18
6	6	Post_06	21
7	7	Post_07	18
8	8	Post_08	20
9	9	Post_09	14
10	10	Post_10	19
11	11	Post_11	16
12	12	Post_12	17
13	13	Post_13	11
14	14	Post_14	0
15	15	Post_15	0

5.2.C - not exists

```sql

-------------------------------
-- PART C: NOT EXISTS, EXCEPT :
-------------------------------

-- Accounts that have never played anything
-- using NOT EXISTS
SELECT p.post_id, p.post_title
FROM posts p
WHERE NOT EXISTS (
    SELECT 1
    FROM likes l
    WHERE p.post_id = l.post_id
);

```

Data Output | Messages | Notifications

SQL

	post_id [PK] integer	post_title character varying (120)
1	15	Post_15
2	14	Post_14

5.2.c - Except

```
74
75
76  -- Accounts that have never played anything
77  -- using EXCEPT
78  SELECT post_id from posts
79  EXCEPT
80  SELECT post_id from likes;
81
82
83
84
```

Data Output | Messages | Notifications

	post_id integer
1	15
2	14

5.3.A - Cross JOIN

```
select posts.post_title, hashtags.hashtag_description
from posts
cross join hashtags

select count(*) from posts;     -- 15
select count(*) from hashtags; -- 5
```

Data Output | Messages | Notifications

Showing r

	post_title character varying (120)	hashtag_description character varying (200)
1	Post_01	Journeys and destinatio...
2	Post_01	Cooking and dining
3	Post_01	Software and hardware
4	Post_01	Training and health
5	Post_01	Artists and releases
6	Post_02	Journeys and destinatio...
7	Post_02	Cooking and dining
8	Post_02	Software and hardware
9	Post_02	Training and health
10	Post_02	Artists and releases
11	Post_03	Journeys and destinatio...
12	Post_03	Cooking and dining
13	Post_03	Software and hardware
14	Post_03	Training and health
15	Post_03	Artists and releases
16	Post_04	Journeys and destinatio...
17	Post_04	Cooking and dining
18	Post_04	Software and hardware
19	Post_04	Training and health
20	Post_04	Artists and releases
21	Post_05	Journeys and destinatio...

Total rows: 75 | Query complete 00:00:00.093

Show that 75 = 15*5

```
select count(*) from posts;     -- 15
select count(*) from hashtags; -- 5
```

Data Output | Messages | Notifications

	count bigint
1	15

```
24  select count(*) from hashtags; -- 5
25
```

Data Output | Messages | Notifications

	count bigint
1	5

5.3.B - Explain output

```
37
38  -- query from 5.1
39  explain
40  select users.user_name, users.email, users.signup_date, 'LIKED' as event,
41         posts.post_title, posts.posted_on, likes.liked_at, posts.author_country, users.country as liker_country
42  from users
43  inner join likes on users.user_id = likes.user_id
44  inner join posts on likes.post_id = posts.post_id
45  order by users.user_name, likes.like_id;
46
```

Data Output | Messages | Notifications

Showing rows: 1 to 13 | Page No: 1

	QUERY PLAN text
1	Sort (cost=35.98..36.48 rows=200 width=1266)
2	Sort Key: users.user_name, likes.like_id
3	-> Nested Loop (cost=12.40..28.34 rows=200 width=1266)
4	-> Hash Join (cost=12.25..17.80 rows=200 width=798)
5	Hash Cond: (likes.user_id = users.user_id)
6	-> Seq Scan on likes (cost=0.00..5.00 rows=200 width=24)
7	-> Hash (cost=11.00..11.00 rows=100 width=782)
8	-> Seq Scan on users (cost=0.00..11.00 rows=100 width=782)
9	-> Memoize (cost=0.15..0.41 rows=1 width=444)
10	Cache Key: likes.post_id
11	Cache Mode: logical
12	-> Index Scan using posts_pkey on posts (cost=0.14..0.40 rows=1 width=444)
13	Index Cond: (post_id = likes.post_id)